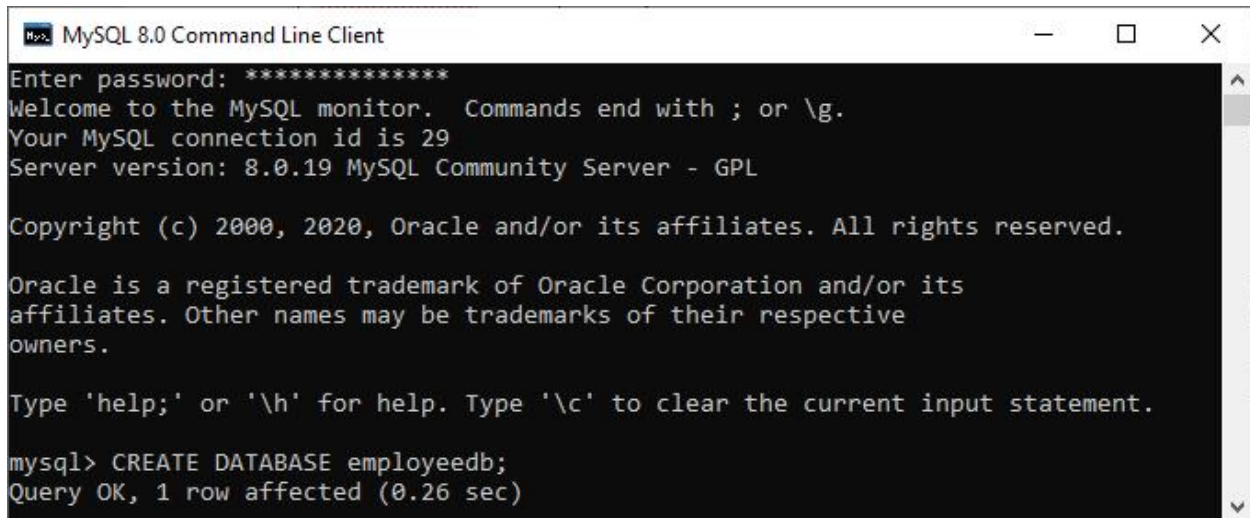


1) MySQL Create Database

MySQL create database is used to create database. For example

create database db1;



```
MySQL 8.0 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 8.0.19 MySQL Community Server - GPL

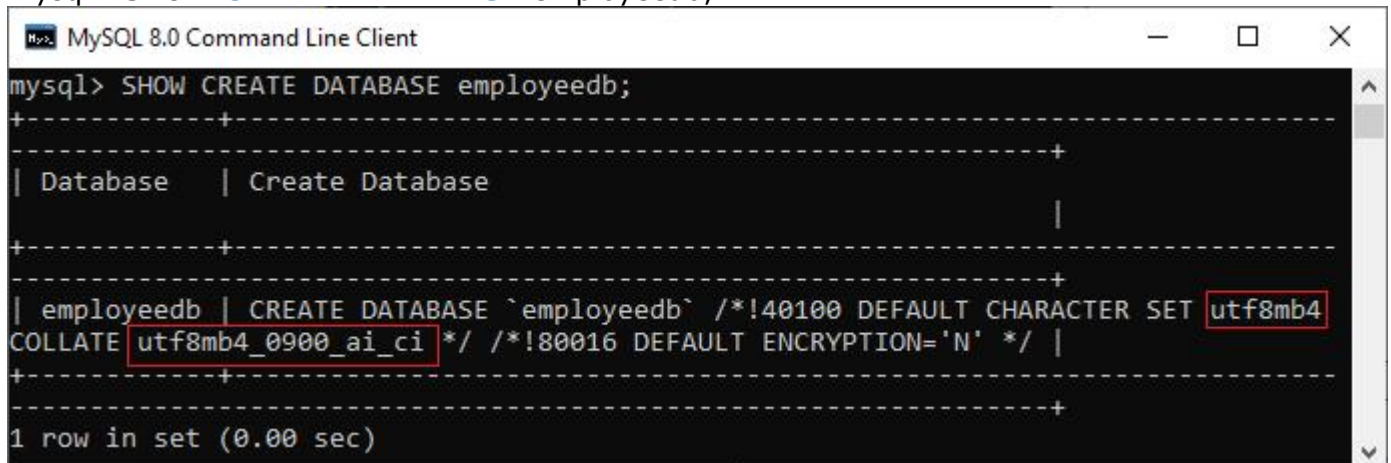
Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE employeeedb;
Query OK, 1 row affected (0.26 sec)
```

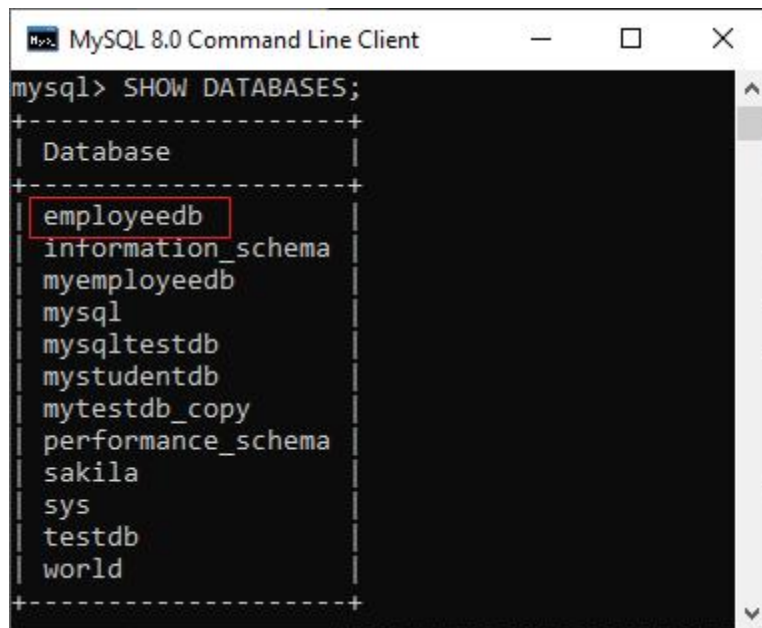
mysql> **SHOW CREATE DATABASE** employeeedb;



```
MySQL 8.0 Command Line Client
mysql> SHOW CREATE DATABASE employeeedb;
+-----+-----+
| Database | Create Database |
+-----+-----+
| employeeedb | CREATE DATABASE `employeeedb` /*!40100 DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci */ /*!80016 DEFAULT ENCRYPTION='N' */ |
+-----+-----+
1 row in set (0.00 sec)
```

mysql> **SHOW DATABASES;**

After executing the above query, we can see all the created databases in the server.



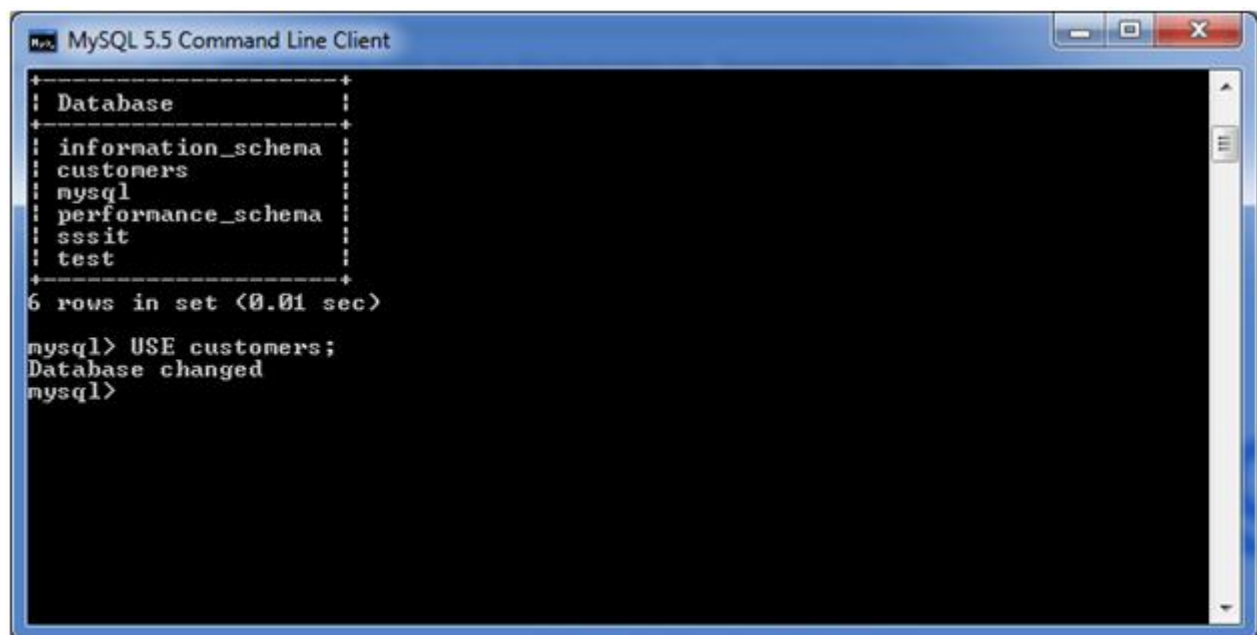
```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| employeedb |
| information_schema |
| myemployeedb |
| mysql |
| mysqltestdb |
| mystudentdb |
| mytestdb_copy |
| performance_schema |
| sakila |
| sys |
| testdb |
| world |
+-----+
```

USE database_name;

Let's take an example to use a database name "customers".

USE customers;

It will look like this:

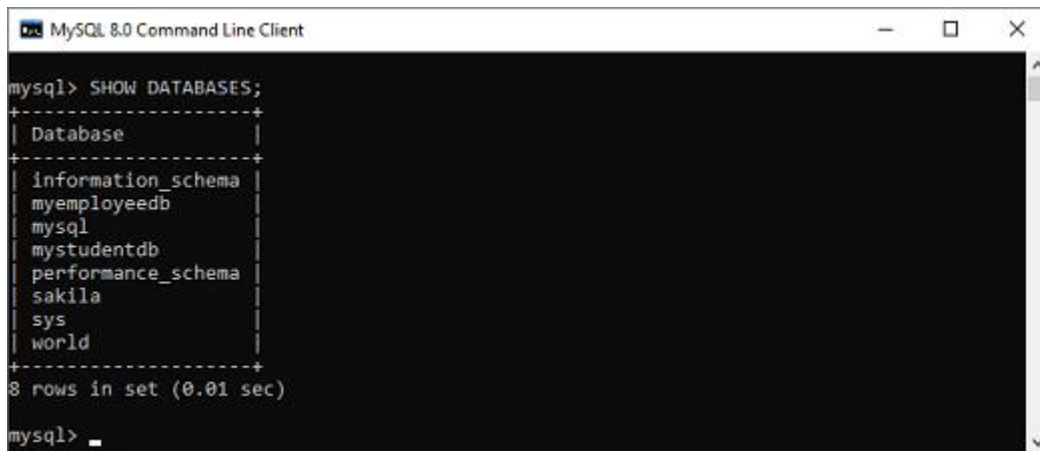


```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| customers |
| mysql |
| performance_schema |
| sssit |
| test |
+-----+
6 rows in set (0.01 sec)

mysql> USE customers;
Database changed
mysql>
```

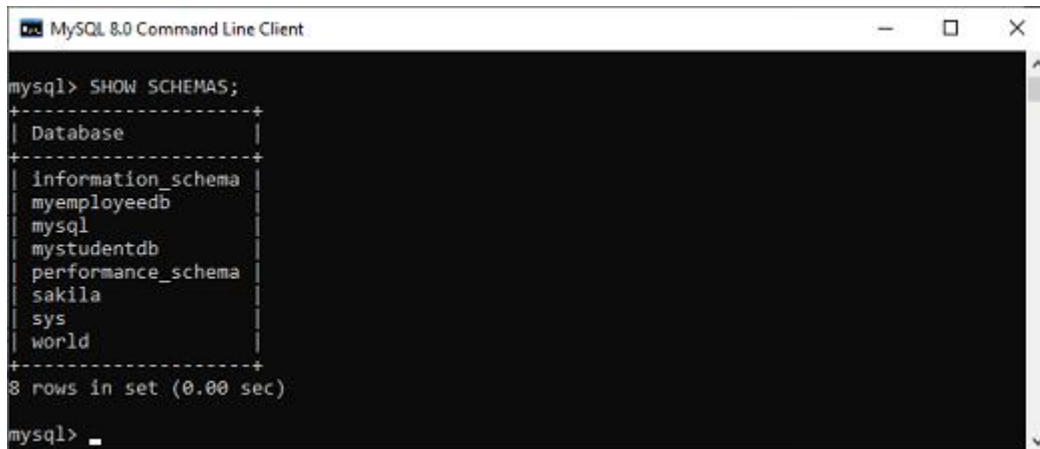
MySQL Show/List Databases

mysql> SHOW DATABASES;



```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| myemployeedb |
| mysql |
| mystudentdb |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
8 rows in set (0.01 sec)

mysql> _
```



```
mysql> SHOW SCHEMAS;
+-----+
| Database |
+-----+
| information_schema |
| myemployeedb |
| mysql |
| mystudentdb |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
8 rows in set (0.00 sec)

mysql> _
```

1. mysql> SHOW DATABASES LIKE "%schema";

The above statement will give the following output:

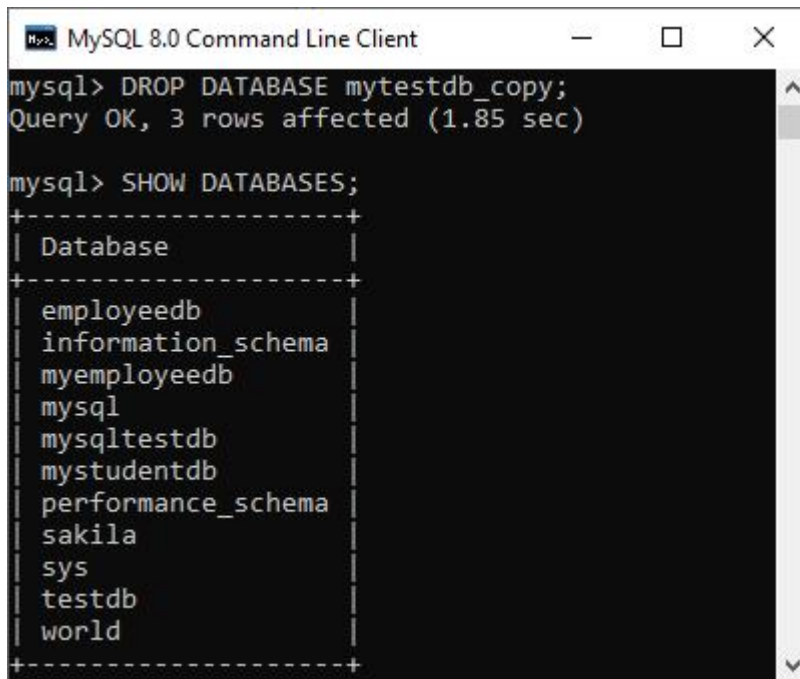


```
mysql> SHOW DATABASES LIKE "%schema";
+-----+
| Database (%schema) |
+-----+
| information_schema |
| performance_schema |
+-----+
2 rows in set (0.00 sec)

mysql> _
```

1. **DROP DATABASE** mytestdb_copy;

Now we can verify that either our database is removed or not by executing the following query. It will look like this:



```
mysql> DROP DATABASE mytestdb_copy;
Query OK, 3 rows affected (1.85 sec)

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| employee |
| information |
| myemployee |
| mysql |
| mysqltest |
| mystudent |
| performance |
| sakila |
| sys |
| test |
| world |
+-----+
```

1. mysql> **CREATE TABLE** employee_table(
2. id **int** NOT NULL AUTO_INCREMENT,
3. **name varchar**(45) NOT NULL,
4. occupation **varchar**(35) NOT NULL,
5. age **int** NOT NULL,
6. **PRIMARY KEY** (id)
7.);

```
MySQL 8.0 Command Line Client

+-----+
| Database |
+-----+
| employeedb |
| information_schema |
| myemployeedb |
| mysql |
| mysqltestdb |
| mystudentdb |
| mytestdb_copy |
| performance_schema |
| sakila |
| sys |
| testdb |
| world |
+-----+
12 rows in set (0.01 sec)

mysql> USE employeedb;
Database changed
mysql> CREATE TABLE employee_table(
  -> id int NOT NULL AUTO_INCREMENT,
  -> name varchar(45) NOT NULL,
  -> occupation varchar(35) NOT NULL,
  -> age int NOT NULL,
  -> PRIMARY KEY (id)
  -> );
Query OK, 0 rows affected (1.47 sec)
```

```
MySQL 8.0 Command Line Client

mysql> SHOW TABLES;
+-----+
| Tables_in_employeedb |
+-----+
| employee_table |
+-----+
1 row in set (0.00 sec)
```

1. mysql> DESCRIBE employee_table;

It will look like this:

```
MySQL 8.0 Command Line Client
mysql> DESCRIBE employee_table;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(45)	NO		NULL	
occupation	varchar(35)	NO		NULL	
age	int	NO		NULL	

1. **ALTER TABLE** cus_tbl
2. **ADD** cus_age **varchar**(40) NOT NULL;

Output:

```
MySQL 5.5 Command Line Client
mysql> SELECT * FROM cus_tbl;
```

cus_id	cus_firstname	cus_surname
5	Ajeet	Maurya
6	Deepika	Chopra
7	Uimal	Jaiswal

```
3 rows in set (0.00 sec)

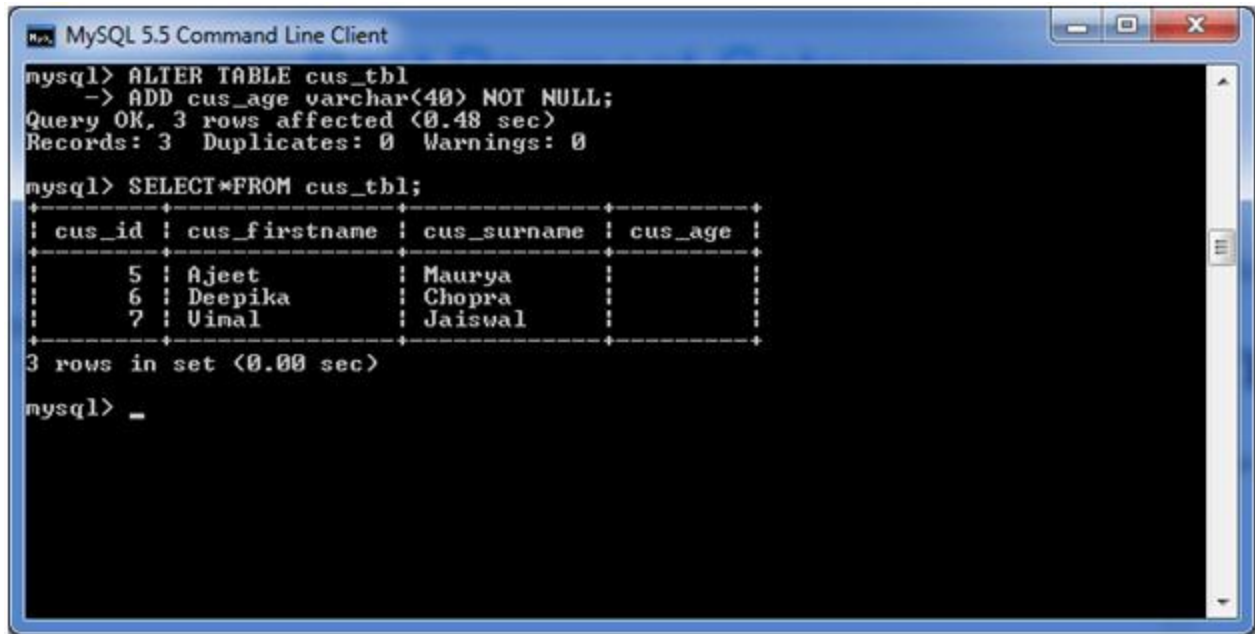
mysql> ALTER TABLE cus_tbl
-> ADD cus_age varchar(40) NOT NULL;
Query OK, 3 rows affected (0.48 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
```

See the recently added column:

SELECT* FROM cus_tbl;

Output:



```
mysql> ALTER TABLE cus_tbl
-> ADD cus_age varchar(40) NOT NULL;
Query OK, 3 rows affected (0.48 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM cus_tbl;
+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_surname | cus_age |
+-----+-----+-----+-----+
| 5 | Ajeet | Maurya |  |
| 6 | Deepika | Chopra |  |
| 7 | Vimal | Jaiswal |  |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

In this example, we add two new columns "cus_address", and cus_salary in the existing table "cus_tbl". cus_address is added after cus_surname column and cus_salary is added after cus_age column.

Use the following query to do this:

1. **ALTER TABLE** cus_tbl
2. **ADD** cus_address **varchar**(100) NOT NULL
3. **AFTER** cus_surname,
4. **ADD** cus_salary **int**(100) NOT NULL
5. **AFTER** cus_age ;

```
MySQL 5.5 Command Line Client
mysql> ALTER TABLE cus_tbl
-> ADD cus_address varchar(100) NOT NULL
-> AFTER cus_surname,
-> ADD cus_salary int(100) NOT NULL
-> AFTER cus_age ;
Query OK, 3 rows affected (0.34 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql>
```

See the recently added columns:

SELECT* FROM cus_tbl;

```
MySQL 5.5 Command Line Client
mysql> ALTER TABLE cus_tbl
-> ADD cus_address varchar(100) NOT NULL
-> AFTER cus_surname,
-> ADD cus_salary int(100) NOT NULL
-> AFTER cus_age ;
Query OK, 3 rows affected (0.34 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> SELECT* FROM cus_tbl;
+-----+-----+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_surname | cus_address | cus_age | cus_salary |
+-----+-----+-----+-----+-----+-----+
| 5 | Ajeet | Maurya |  |  | 0 |
| 6 | Deepika | Chopra |  |  | 0 |
| 7 | Vimal | Jaiswal |  |  | 0 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)
mysql> _
```

3) MODIFY column in the table

1. **ALTER TABLE** cus_tbl
2. **MODIFY** cus_surname **varchar**(50) NULL;

```
MySQL 5.5 Command Line Client

+----+-----+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_surname | cus_address | cus_age | cus_salary |
+----+-----+-----+-----+-----+-----+
| 5 | Ajeet | Maurya | | | 0 |
| 6 | Deepika | Chopra | | | 0 |
| 7 | Vimal | Jaiswal | | | 0 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> ALTER TABLE cus_tbl
-> MODIFY cus_surname varchar(50) NULL;
Query OK, 3 rows affected (0.33 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> _
```

See the table structure:

```
MySQL 5.5 Command Line Client

Records: 3 Duplicates: 0 Warnings: 0
mysql> SELECT* FROM cus_tbl;

+----+-----+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_surname | cus_address | cus_age | cus_salary |
+----+-----+-----+-----+-----+-----+
| 5 | Ajeet | Maurya | | | 0 |
| 6 | Deepika | Chopra | | | 0 |
| 7 | Vimal | Jaiswal | | | 0 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> DESCRIBE cus_tbl;

+----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| cus_id | int(11) | NO | PRI | NULL | auto_increment |
| cus_firstname | varchar(100) | NO | | NULL | |
| cus_surname | varchar(50) | YES | | NULL | |
| cus_address | varchar(100) | NO | | NULL | |
| cus_age | varchar(40) | NO | | NULL | |
| cus_salary | int(100) | NO | | NULL | |
+----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

1. **ALTER TABLE** cus_tbl
2. CHANGE **COLUMN** cus_surname cus_title
3. **varchar**(20) NOT NULL;

Output:

```
MySQL 5.5 Command Line Client
mysql> SELECT*FROM cus_tbl;
+-----+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_surname | cus_age | cus_salary |
+-----+-----+-----+-----+-----+
| 5 | Ajeet | Maurya |  | 0 |
| 6 | Deepika | Chopra |  | 0 |
| 7 | Vimal | Jaiswal |  | 0 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> ALTER TABLE cus_tbl
-> CHANGE COLUMN cus_surname cus_title
-> varchar(20) NOT NULL;
Query OK, 3 rows affected (0.25 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> _
```

1. **ALTER TABLE** cus_tbl
2. **RENAME TO** cus_table;

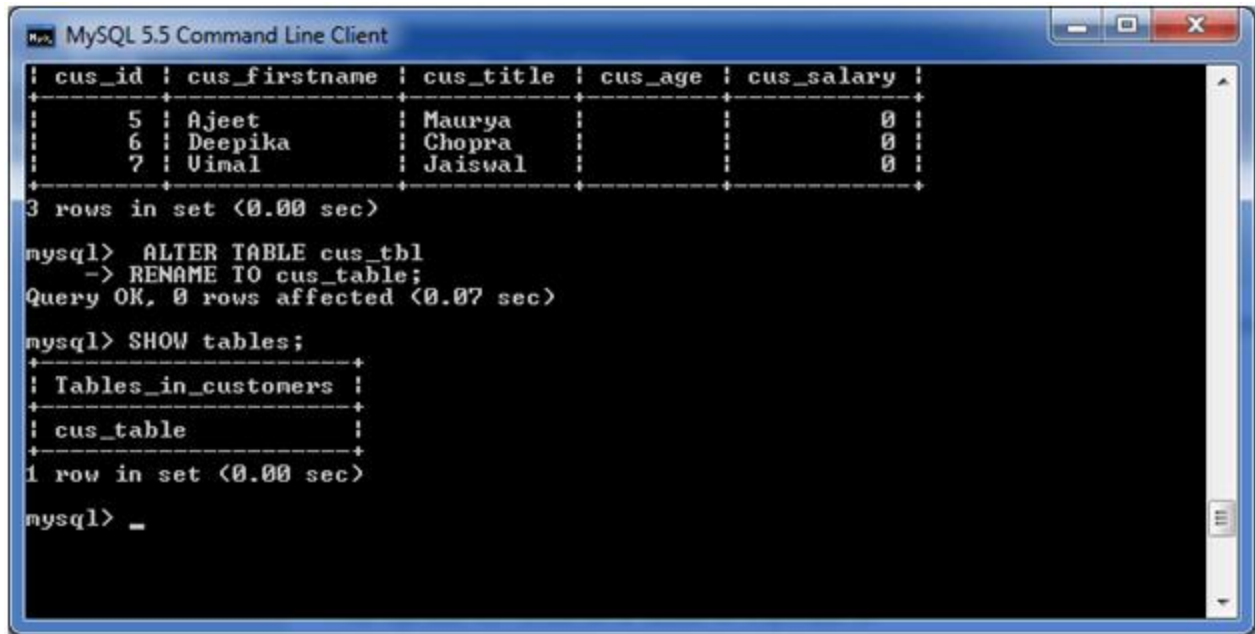
Output:

```
MySQL 5.5 Command Line Client
mysql> SELECT*FROM cus_tbl;
+-----+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_title | cus_age | cus_salary |
+-----+-----+-----+-----+-----+
| 5 | Ajeet | Maurya |  | 0 |
| 6 | Deepika | Chopra |  | 0 |
| 7 | Vimal | Jaiswal |  | 0 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> ALTER TABLE cus_tbl
-> RENAME TO cus_table;
Query OK, 0 rows affected (0.07 sec)

mysql>
```

See the renamed table:



```
MySQL 5.5 Command Line Client
+----+-----+-----+-----+-----+
| cus_id | cus_firstname | cus_title | cus_age | cus_salary |
+----+-----+-----+-----+-----+
| 5 | Ajeet | Maurya |  | 0 |
| 6 | Deepika | Chopra |  | 0 |
| 7 | Vimal | Jaiswal |  | 0 |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

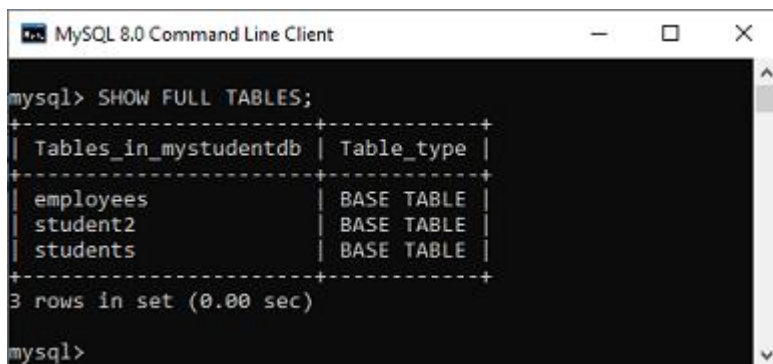
mysql> ALTER TABLE cus_tbl
-> RENAME TO cus_table;
Query OK, 0 rows affected (0.07 sec)

mysql> SHOW tables;
+-----+
| Tables_in_customers |
+-----+
| cus_table |
+-----+
1 row in set (0.00 sec)

mysql> _
```

1. mysql> SHOW **FULL** TABLES;

This statement will give the following output:



```
MySQL 8.0 Command Line Client

mysql> SHOW FULL TABLES;
+-----+-----+
| Tables_in_mystudentdb | Table_type |
+-----+-----+
| employees               | BASE TABLE |
| student2                | BASE TABLE |
| students                 | BASE TABLE |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

mysql> rename table student1 to stud;

Query OK, 0 rows affected (0.11 sec)

mysql> show tables ;

```
| order1      |
| order2      |
```

| products |

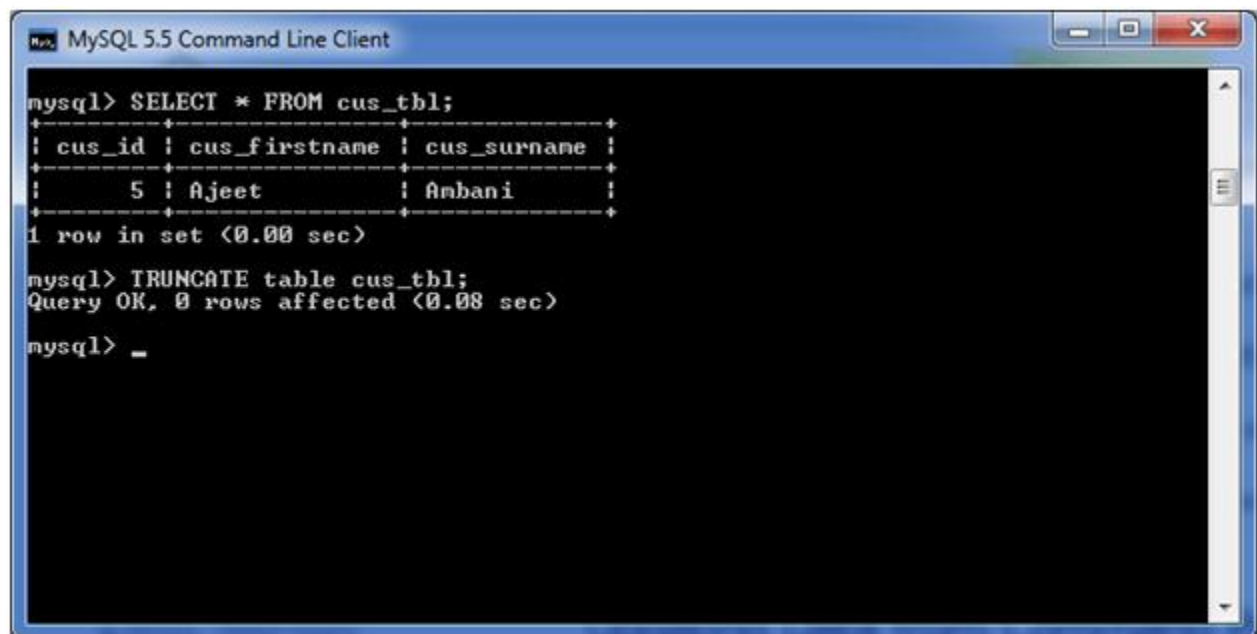
| stud |

| student |

+-----+

7 rows in set (0.02 sec)

TRUNCATE TABLE cus_tbl;



```
mysql> SELECT * FROM cus_tbl;
+-----+-----+-----+
| cus_id | cus_firstname | cus_surname |
+-----+-----+-----+
|      5 | Ajeet        | Ambani      |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> TRUNCATE table cus_tbl;
Query OK, 0 rows affected (0.08 sec)

mysql> _
```

1. **SELECT* FROM** cus_tbl;

Output:

```
MySQL 5.5 Command Line Client
1 row in set (0.00 sec)
mysql> TRUNCATE table cus_tbl;
Query OK, 0 rows affected (0.08 sec)
mysql> SELECT* FROM cus_tbl;
Empty set (0.00 sec)
mysql>
```

```
MySQL 8.0 Command Line Client
mysql> SHOW COLUMNS FROM customer;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| cust_id    | int           | NO   | PRI | NULL    |       |
| name       | varchar(35)   | YES  |     | NULL    |       |
| occupation | varchar(25)   | YES  |     | NULL    |       |
| age        | int           | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

1. mysql> EXPLAIN **SELECT** * **FROM** customer;

```
MySQL 8.0 Command Line Client
mysql> EXPLAIN SELECT * FROM customer;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table  | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | customer | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 7    | 100.00  | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.12 sec)
```

```
MySQL 8.0 Command Line Client
mysql> DROP TABLE orders;
ERROR 1051 (42502): Unknown table 'mystudentdb.orders'
mysql>
```

```
MySQL 8.0 Command Line Client
Server version: 8.0.19 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mystudentdb;
Database changed
mysql> CREATE TEMPORARY TABLE Students( student_name VARCHAR(40) NOT NULL, total_marks DECIMAL(12,2) NOT NULL DEFAULT 0.00,
total_subjects INT UNSIGNED NOT NULL DEFAULT 0);
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO Students(student_name, total_marks, total_subjects) VALUES ('Joseph', 150.75, 2), ('Peter', 180.75, 2);
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
```

1. **CREATE TABLE** IF NOT EXISTS duplicate_table
2. **SELECT** * **FROM** original_table **WHERE** Year = '2016';

This statement creates a duplicate table that contains data for the **year 2016** only. We can verify the table using a SELECT statement, as shown below:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE IF NOT EXISTS duplicate_table
-> SELECT * FROM original_table WHERE Year='2016';
Query OK, 3 rows affected (0.99 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM duplicate_table;
+----+-----+-----+-----+-----+
| Id | Name  | Product | Country | Year |
+----+-----+-----+-----+-----+
| 2  | Joseph | Laptop  | India   | 2016 |
| 3  | John   | TV      | USA     | 2016 |
| 6  | Peter  | Mouse   | England | 2016 |
+----+-----+-----+-----+-----+
```

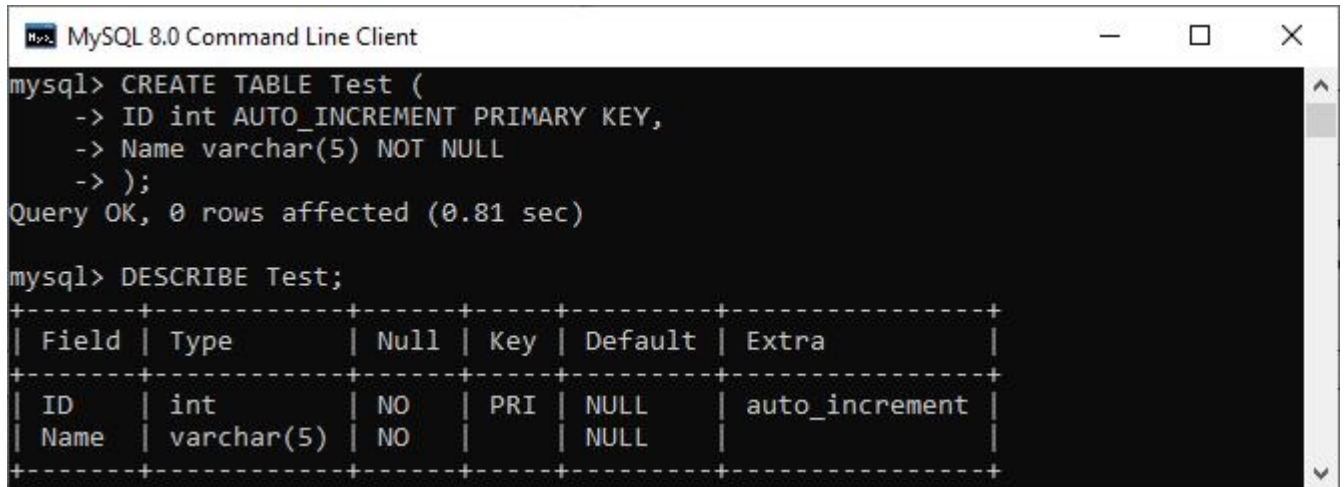
Suppose there a need to copy an existing table along with all dependent objects associated with the table, execute the two statements that are given below:

MySQL ADD COLUMN Example

Let us understand it with the help of various examples. Here, we will create a table named **"Test"** using the following statements:

1. **CREATE TABLE** Test (
2. Stude_id **int** AUTO_INCREMENT **PRIMARY KEY**,
3. **Name varchar**(55) NOT NULL
4.);

The table structure looks like the below image:



```
mysql> CREATE TABLE Test (  
  -> ID int AUTO_INCREMENT PRIMARY KEY,  
  -> Name varchar(55) NOT NULL  
  -> );  
Query OK, 0 rows affected (0.81 sec)  
  
mysql> DESCRIBE Test;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type          | Null | Key | Default | Extra          |  
+-----+-----+-----+-----+-----+-----+  
| ID    | int           | NO   | PRI | NULL    | auto_increment |  
| Name  | varchar(55)   | NO   |     | NULL    |                |  
+-----+-----+-----+-----+-----+-----+
```

After creating a table, we want to add a new column named City to the Test table. Since we have not specified the new column position explicitly after the column name, MySQL will add it as the last column.

1. **ALTER TABLE** Test
2. **ADD COLUMN** City **VARCHAR**(30) NOT NULL;

Next, we want to add a new column named **Phone_number** to the **Test** table. This time, we will explicitly specify the new column position so that MySQL adds the column to the specified place.

1. **ALTER TABLE** Test
2. **ADD COLUMN** Phone_number **VARCHAR**(20) NOT NULL ;

In the below output, we can see that the two columns are added successfully at the specified position.

```
MySQL 8.0 Command Line Client

mysql> ALTER TABLE Test
  -> ADD COLUMN City VARCHAR(30) NOT NULL;
Query OK, 0 rows affected (0.77 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Test
  -> ADD COLUMN Phone_number VARCHAR(20) NOT NULL AFTER Name;
Query OK, 0 rows affected (1.42 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> DESCRIBE Test;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| ID         | int           | NO   | PRI | NULL    | auto_increment |
| Name       | varchar(5)    | NO   |     | NULL    |                 |
| Phone_number | varchar(20)   | NO   |     | NULL    |                 |
| City       | varchar(30)   | NO   |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
```

Let us add some data into the Test table using the [INSERT statement](#) as follows:

1. **INSERT INTO** Test(**Name**, Phone_number, City)
2. **VALUES** ('Peter', '34556745362', 'California'),
3. ('Mike', '983635674562', 'Texas');

It will look like this.

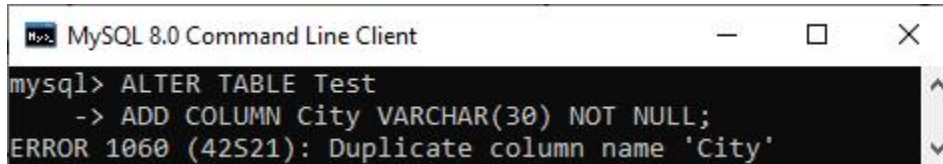
```
MySQL 8.0 Command Line Client

mysql> INSERT INTO Test( Name, Phone_number, City)
  -> VALUES ('Peter', '34556745362', 'California'),
  -> ('Mike', '983635674562', 'Texas');
Query OK, 2 rows affected (0.14 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Test;
+-----+-----+-----+-----+
| Stude_id | Name  | Phone_number | City      |
+-----+-----+-----+-----+
|         1 | Peter | 34556745362  | California |
|         2 | Mike  | 983635674562 | Texas     |
+-----+-----+-----+-----+
```

1. **ALTER TABLE** Test
2. **ADD COLUMN** City **VARCHAR**(30) NOT NULL;

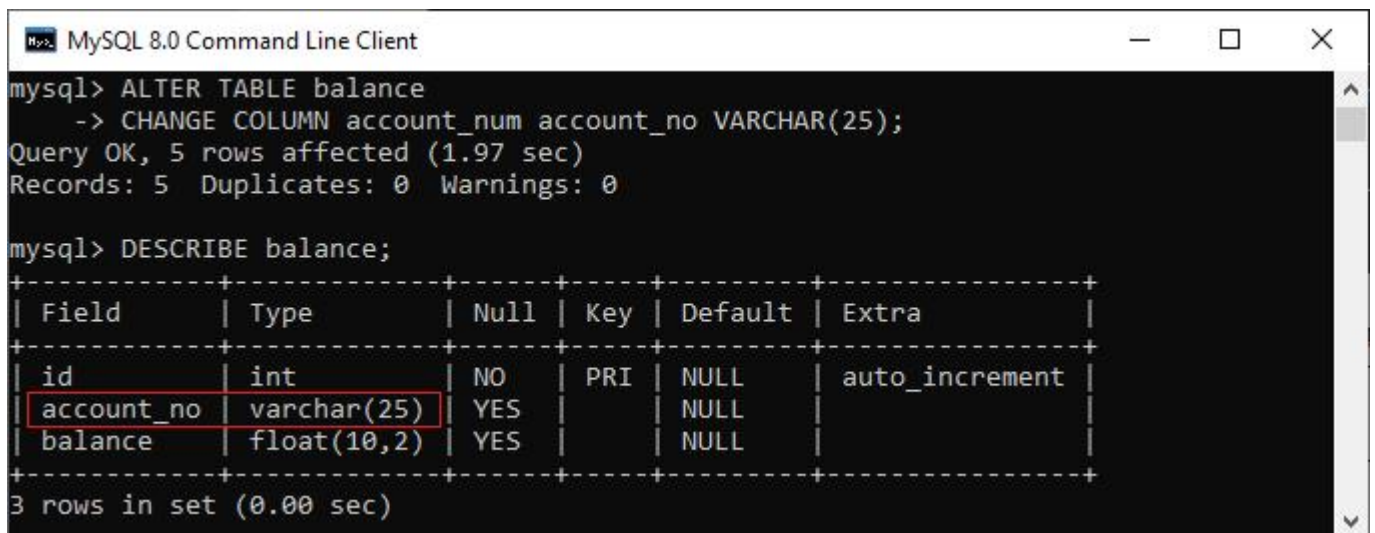
We will get the following error message.



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE Test
  -> ADD COLUMN City VARCHAR(30) NOT NULL;
ERROR 1060 (42S21): Duplicate column name 'City'
```

1. mysql> **ALTER TABLE** balance
2. CHANGE **COLUMN** account_num account_no **VARCHAR**(25);

After executing the above command, we can verify it by using the DESCRIBE statement again. In the below image, the column name account_num and its data type have changed successfully.



```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE balance
  -> CHANGE COLUMN account_num account_no VARCHAR(25);
Query OK, 5 rows affected (1.97 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> DESCRIBE balance;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL    | auto_increment |
| account_no | varchar(25)   | YES  |     | NULL    |                 |
| balance    | float(10,2)   | YES  |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

1. mysql> **ALTER TABLE** customer
2. CHANGE id cust_id **int**,
3. CHANGE customer_name cust_name **varchar**(45);

After executing the above command, we can verify it by using the SELECT statement again. In the below image, the column name id and customer_name have changed successfully:

```
MySQL 8.0 Command Line Client
mysql> ALTER TABLE customer
-> CHANGE id cust_id int,
-> CHANGE customer_name cust_name varchar(45);
Query OK, 5 rows affected (1.39 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM customer;
+-----+-----+-----+-----+
| cust_id | cust_name | account_no | email |
+-----+-----+-----+-----+
| 1 | Stephen | 1030 | stephen@javatpoint.com |
| 2 | Jenifer | 2035 | jenifer@javatpoint.com |
| 3 | Mathew | 5564 | mathew@javatpoint.com |
| 4 | Smith | 4534 | smith@javatpoint.com |
| 5 | david | 7648 | david@javatpoint.com |
+-----+-----+-----+-----+
```

MySQL View

A view is a database object that has no values. Its contents are based on the base table. It contains rows and columns similar to the real table. In MySQL, the View is a **virtual table** created by a query by joining one or more tables. It is operated similarly to the base table but does not contain any data of its own. The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.

Syntax

Following is the syntax to create a view in MySQL:

```
CREATE [OR REPLACE] VIEW view_name AS
SELECT columns
FROM tables
[WHERE conditions];
```

```
CREATE VIEW trainer AS
SELECT course_name, trainer
FROM courses;
```

```
MySQL 8.0 Command Line Client
mysql> SHOW TABLES;
+-----+
| Tables_in_testdb |
+-----+
| contact          |
| courses           |
| customer          |
+-----+
3 rows in set (0.13 sec)

mysql> CREATE VIEW trainer AS
-> SELECT course_name, trainer
-> FROM courses;
Query OK, 0 rows affected (0.33 sec)
```

SELECT * **FROM** trainer;

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM trainer;
+-----+-----+
| course_name | trainer |
+-----+-----+
| Java        | Mike    |
| Python      | James   |
| Android     | Robin   |
| Hadoop      | Stephen |
| Testing     | Micheal |
+-----+-----+
5 rows in set (0.05 sec)
```

ALTER VIEW trainer **AS**

SELECT id, course_name, trainer

FROM courses;

```
MySQL 8.0 Command Line Client
mysql> ALTER VIEW trainer AS
-> SELECT id, course_name, trainer
-> FROM courses;
Query OK, 0 rows affected (0.22 sec)

mysql> SELECT * FROM trainer;
+----+-----+-----+
| id | course_name | trainer |
+----+-----+-----+
| 1  | Java       | Mike   |
| 2  | Python     | James  |
| 3  | Android    | Robin  |
| 4  | Hadoop     | Stephen|
| 5  | Testing    | Micheal|
+----+-----+-----+
```

1. **DROP VIEW** trainer;

After successful execution, it is required to verify the view is available or not as below:

```
MySQL 8.0 Command Line Client
mysql> DROP VIEW trainer;
Query OK, 0 rows affected (0.18 sec)

mysql> SELECT * FROM trainer;
ERROR 1146 (42S02): Table 'testdb.trainer' doesn't exist
```

5) MySQL Insert Query

MySQL insert query is used to insert records into table. For example:

```
insert into customers values(101,'rahul','delhi');
```

6) MySQL Update Query

MySQL update query is used to update records of a table. For example:

```
update customers set name='bob', city='london' where id=101;
```

7) MySQL Delete Query

MySQL update query is used to delete records of a table from database. For example:

delete from customers **where** id=101;

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE ShirtBrands(Id INTEGER, BrandName VARCHAR(40) UNIQUE, Size VARCHAR(30));
Query OK, 0 rows affected (0.88 sec)

mysql> INSERT INTO ShirtBrands(Id, BrandName, Size) VALUES(1, 'Pantaloon', 38), (2, 'Cantabil', 40);
Query OK, 2 rows affected (0.26 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO ShirtBrands(Id, BrandName, Size) VALUES(3, 'Raymond', 38), (4, 'Cantabil', 40);
ERROR 1062 (23000): Duplicate entry 'Cantabil' for key 'shirtbrands.BrandName'
```

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE Persons (
->   ID int NOT NULL,
->   Name varchar(45) NOT NULL,
->   Age int CHECK (Age>=18)
-> );
Query OK, 0 rows affected (0.87 sec)

mysql> INSERT INTO Persons(Id, Name, Age)
-> VALUES (1,'Robert', 28),
-> (2, 'Joseph', 35),
-> (3, 'Peter', 40);
Query OK, 3 rows affected (0.30 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Persons(Id, Name, Age) VALUES (1,'Robert', 15);
ERROR 3819 (HY000): Check constraint 'persons_chk_1' is violated.
```

1. mysql> **CREATE TABLE** Persons (
2. ID **int** NOT NULL,
3. **Name varchar**(45) NOT NULL,
4. Age **int**,
5. City **varchar**(25) **DEFAULT** 'New York'
6.);
7. mysql> **INSERT INTO** Persons(Id, **Name**, Age, City)
8. **VALUES** (1,'Robert', 15, 'Florida'),
9. (2, 'Joseph', 35, 'California'),
10. (3, 'Peter', 40, 'Alaska');

```
11.mysql> CREATE TABLE Animals(  
12.id int NOT NULL AUTO_INCREMENT,  
13.name CHAR(30) NOT NULL,  
14.PRIMARY KEY (id));  
  
15.mysql> INSERT INTO Animals (name) VALUES  
16.('Tiger'),('Dog'),('Penguin'),  
17.('Camel'),('Cat'),('Ostrich');
```

Table: Persons

```
1. CREATE TABLE Persons (  
2.     Person_ID int NOT NULL PRIMARY KEY,  
3.     Name varchar(45) NOT NULL,  
4.     Age int,  
5.     City varchar(25)  
6. );
```

Table: Orders

```
1. CREATE TABLE Orders (  
2.     Order_ID int NOT NULL PRIMARY KEY,  
3.     Order_Num int NOT NULL,  
4.     Person_ID int,  
5.     FOREIGN KEY (Person_ID) REFERENCES Persons(Person_ID)  
6. );
```

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM Persons;
+-----+-----+-----+-----+
| Person_ID | Name   | Age  | City   |
+-----+-----+-----+-----+
|          1 | Stephen | 15   | Florida |
|          2 | Joseph  | 35   | California |
|          3 | Peter   | 40   | Alaska  |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Orders;
+-----+-----+-----+
| Order_ID | Order_Num | Person_ID |
+-----+-----+-----+
|          1 | 5544      | 2         |
|          2 | 3322      | 3         |
|          3 | 2135      | 2         |
|          4 | 3432      | 1         |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

1. **UPDATE** trainer
2. **SET** email = 'mike@tutorialandexamples.com'
3. **WHERE** course_name = 'Java';

After successful execution, we will verify the table using the below statement:

1. **SELECT * FROM** trainer;

In the output, we can see that our table is updated as per our conditions.

```
MySQL 8.0 Command Line Client
mysql> UPDATE trainer
  -> SET email = 'mike@tutorialandexamples.com'
  -> WHERE course_name = 'Java';
Query OK, 1 row affected (0.26 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM trainer;
+-----+-----+-----+
| course_name | trainer | email                               |
+-----+-----+-----+
| Java        | Mike    | mike@tutorialandexamples.com       |
| Python      | James   | james@javatpoint.com               |
| Android     | Robin   | robin@javatpoint.com               |
| Hadoop      | Stephen | stephen@javatpoint.com             |
| Testing     | Micheal | micheal@javatpoint.com             |
+-----+-----+-----+
```

```
MySQL 8.0 Command Line Client
mysql> UPDATE People
  -> SET name = 'Mary', occupation = 'Content Writer'
  -> WHERE id = 105;
Query OK, 1 row affected (0.64 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM People;
+----+-----+-----+-----+
| id | name   | occupation | age |
+----+-----+-----+-----+
| 101 | Peter  | Engineer   | 32  |
| 102 | Joseph | Developer  | 30  |
| 103 | Mike   | Leader     | 28  |
| 104 | Stephen | Scientist  | 45  |
| 105 | Mary   | Content Writer | NULL |
| 106 | Bob    | Actor      | NULL |
+----+-----+-----+-----+
```

1. mysql> **DELETE FROM** Employees **WHERE** emp_id=107;

After the execution of the query, it will return the output as below image. Once the record is deleted, verify the table using the **SELECT statement**:

```
MySQL 8.0 Command Line Client
mysql> DELETE FROM Employees WHERE emp_id=107;
Query OK, 1 row affected (0.10 sec)

mysql> SELECT * FROM Employees;
+-----+-----+-----+-----+-----+
| emp_id | name   | birthdate | gender | hire_date |
+-----+-----+-----+-----+-----+
| 101    | Bryan | 1988-08-12 | M      | 2015-08-26 |
| 102    | Joseph | 1978-05-12 | M      | 2014-10-21 |
| 103    | Mike   | 1984-10-13 | M      | 2017-10-28 |
| 104    | Daren  | 1979-04-11 | F      | 2006-11-01 |
| 105    | Marie  | 1990-02-11 | F      | 2018-10-12 |
| 106    | Marco  | 1988-04-11 | M      | 2010-10-12 |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

If we want to delete all records from the table, there is no need to use the WHERE clause with the DELETE statement. See the below code and output:

```
MySQL 8.0 Command Line Client
mysql> DELETE FROM Employees;
Query OK, 6 rows affected (0.12 sec)

mysql> SELECT * FROM Employees;
Empty set (0.00 sec)
```

1. `mysql> SELECT Name, Email, City FROM employee_detail;`

```
MySQL 8.0 Command Line Client
mysql> SELECT Name, Email, City FROM employee_detail;
+-----+-----+-----+
| Name   | Email                               | City      |
+-----+-----+-----+
| Peter  | peter@javatpoint.com               | Texas     |
| Suzi   | suzi@javatpoint.com                | California |
| Joseph | joseph@javatpoint.com               | Alaska    |
| Alex   | alex@javatpoint.com                | Los Angeles |
| Mark   | mark@javatpoint.com                 | Washington |
| Stephen | stephen@javatpoint.com              | New York  |
+-----+-----+-----+
```

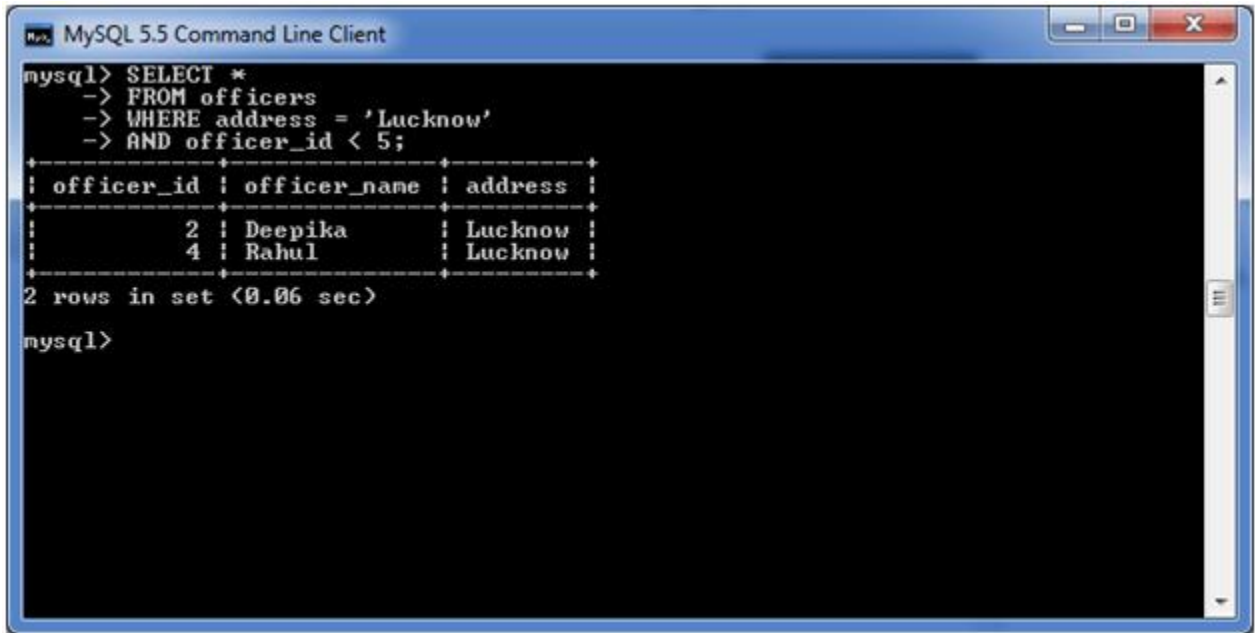
MySQL WHERE Clause with AND condition

In this example, we are retrieving data from the table "officers" with AND condition.

Execute the following query:

1. **SELECT** *
2. **FROM** officers
3. **WHERE** address = 'Lucknow'
4. **AND** officer_id < 5;

Output:



```
mysql> SELECT *
-> FROM officers
-> WHERE address = 'Lucknow'
-> AND officer_id < 5;
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 2 | Deepika | Lucknow |
| 4 | Rahul | Lucknow |
+-----+-----+-----+
2 rows in set (0.06 sec)

mysql>
```

WHERE Clause with OR condition

Execute the following query:

1. **SELECT** *
2. **FROM** officers
3. **WHERE** address = 'Lucknow'
4. **OR** address = 'Mau';

Output:

```
MySQL 5.5 Command Line Client
mysql> SELECT *
-> FROM officers
-> WHERE address = 'Lucknow'
-> OR address = 'Mau';
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 4 | Rahul | Lucknow |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

SELECT DISTINCT address

FROM officers;

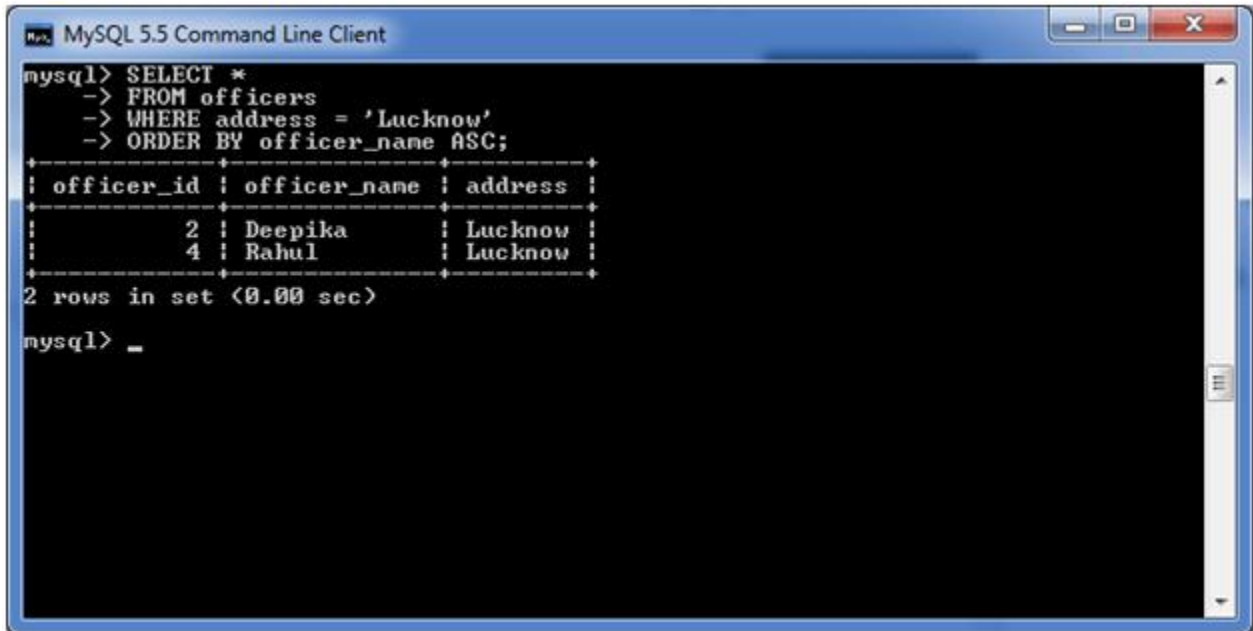
```
MySQL 5.5 Command Line Client
mysql> SELECT * FROM officers;
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 3 | Uimal | Faizabad |
| 4 | Rahul | Lucknow |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT DISTINCT address
-> FROM officers;
+-----+
| address |
+-----+
| Mau |
| Lucknow |
| Faizabad |
+-----+
3 rows in set (0.00 sec)

mysql> _
```

1. **SELECT** * **FROM** officers
2. **WHERE** address = 'Lucknow'
3. **ORDER BY** officer_name;

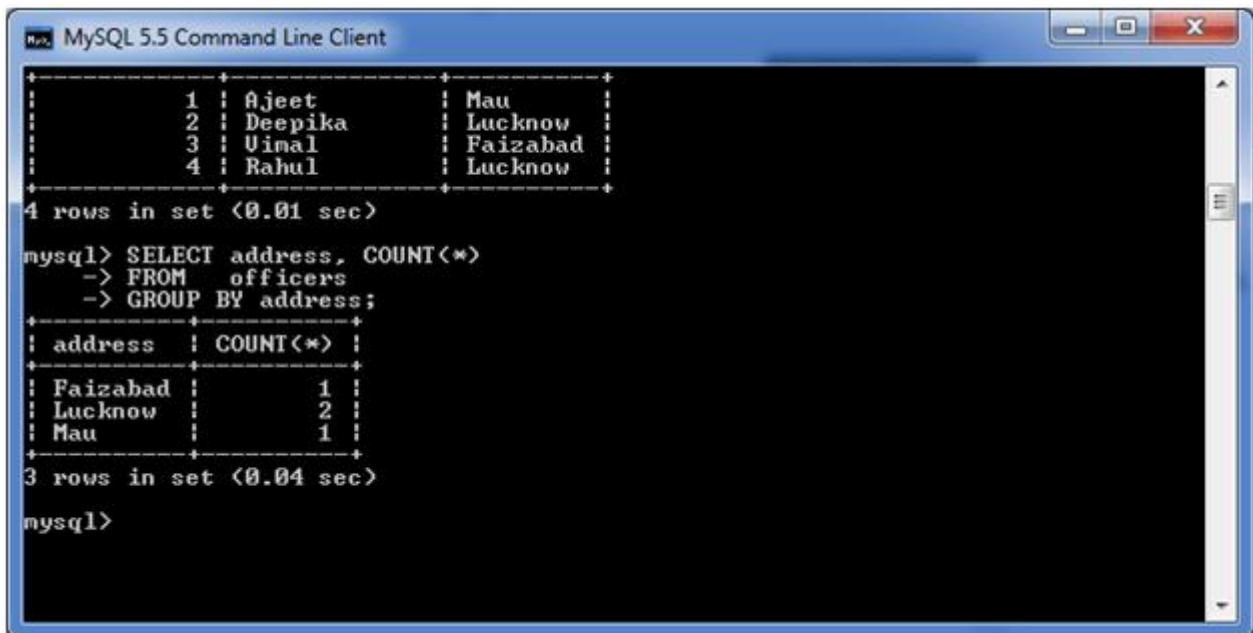
1. **SELECT** * **FROM** officers
2. **WHERE** address = 'Lucknow'
3. **ORDER BY** officer_name **ASC**;



```
mysql> SELECT *  
-> FROM officers  
-> WHERE address = 'Lucknow'  
-> ORDER BY officer_name ASC;  
+-----+-----+-----+  
| officer_id | officer_name | address |  
+-----+-----+-----+  
| 2 | Deepika | Lucknow |  
| 4 | Rahul | Lucknow |  
+-----+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql> _
```

1. **SELECT** address, **COUNT**(*) **FROM** officers
2. **GROUP BY** address;

Output:



```
mysql> SELECT address, COUNT(*)  
-> FROM officers  
-> GROUP BY address;  
+-----+-----+  
| address | COUNT(*) |  
+-----+-----+  
| Faizabad | 1 |  
| Lucknow | 2 |  
| Mau | 1 |  
+-----+-----+  
3 rows in set (0.04 sec)  
  
mysql>
```

1. **SELECT** emp_name, **SUM**(working_hours) **AS** "Total working hours"
2. **FROM** employees **GROUP BY** emp_name
3. **HAVING SUM**(working_hours) > 5;

```

MySQL 5.5 Command Line Client
10 rows in set (0.00 sec)

mysql> SELECT emp_name, SUM(working_hours) AS "Total working hours"
-> FROM employees
-> GROUP BY emp_name
-> HAVING SUM(working_hours) > 5;
+-----+-----+
| emp_name | Total working hours |
+-----+-----+
| Ajeet    | 36                  |
| Ayan     | 20                  |
| Milan    | 27                  |
| Ruchi    | 12                  |
+-----+-----+
4 rows in set (0.00 sec)

mysql> _

```

SELECT SUM(working_hours) **AS** "Total working hours" **FROM** employees;

mysql> **SELECT AVG**(working_hours) Avg_working_hours **FROM** employees;

mysql> **SELECT MIN**(income) **AS** Minimum_Income **FROM** employees;

Output

The above query produces the result of minimum values in all rows. After execution, we will get the output as below:

```

MySQL 8.0 Command Line Client

mysql> SELECT MIN(income) AS Minimum_Income FROM employees;
+-----+
| Minimum_Income |
+-----+
| 70000          |
+-----+
1 row in set (0.00 sec)

```

1. mysql> **SELECT MAX**(income) **AS** "Maximum Income" **FROM** employees;

Output

The above query produces the result of maximum values in all rows. After execution, we will get the output as below:

```
MySQL 8.0 Command Line Client
mysql> SELECT MAX(income) AS "Maximum Income" FROM employees;
+-----+
| Maximum Income |
+-----+
|          5000000 |
+-----+
1 row in set (0.00 sec)
```

1. mysql> **SELECT** city, **MAX**(income) **AS** "Maximum Income"
2. **FROM** employees
3. **GROUP BY** city
4. **HAVING MAX**(income) >= 200000;

```
MySQL 8.0 Command Line Client
mysql> SELECT city, MAX(income) AS "Maximum Income"
-> FROM employees
-> GROUP BY city
-> HAVING MAX(income) >= 200000;
+-----+-----+
| city      | Maximum Income |
+-----+-----+
| Newyork   |          200000 |
| California|          5000000 |
| Arizona   |          1000000 |
| Florida   |          5000000 |
| Georgia   |          250000  |
| Alaska    |          450000  |
+-----+-----+
```

6. MySQL MAX() Function in Subquery Example

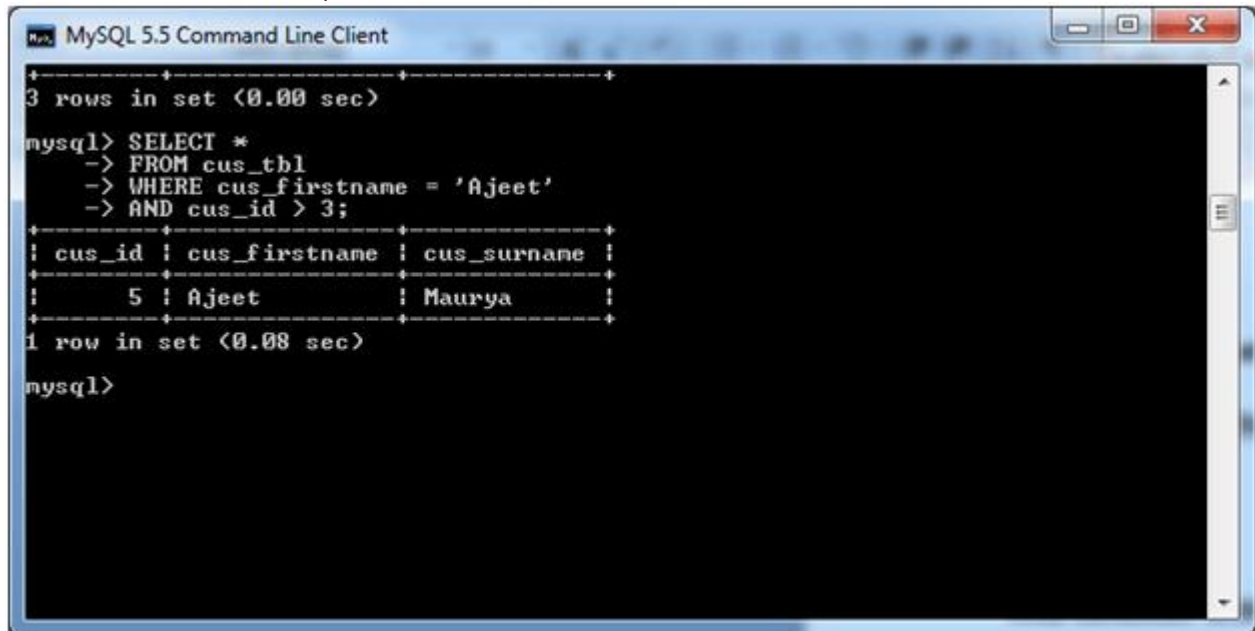
Sometimes it is required to use the subquery for returning the maximum value in the table. In that case, we use the following query:

1. mysql> **SELECT** * **FROM** employees **WHERE**
2. emp_age = (**SELECT MAX**(emp_age) **FROM** employees);

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM employees WHERE
-> emp_age = (SELECT MAX(emp_age) FROM employees);
+-----+-----+-----+-----+-----+
| emp_id | emp_name | emp_age | city      | income |
+-----+-----+-----+-----+-----+
|      106 | Kane     |      45 | Alaska    | 450000  |
|      111 | Alexander|      45 | California| 70000   |
+-----+-----+-----+-----+-----+
```

1. **SELECT** *
2. **FROM** cus_tbl

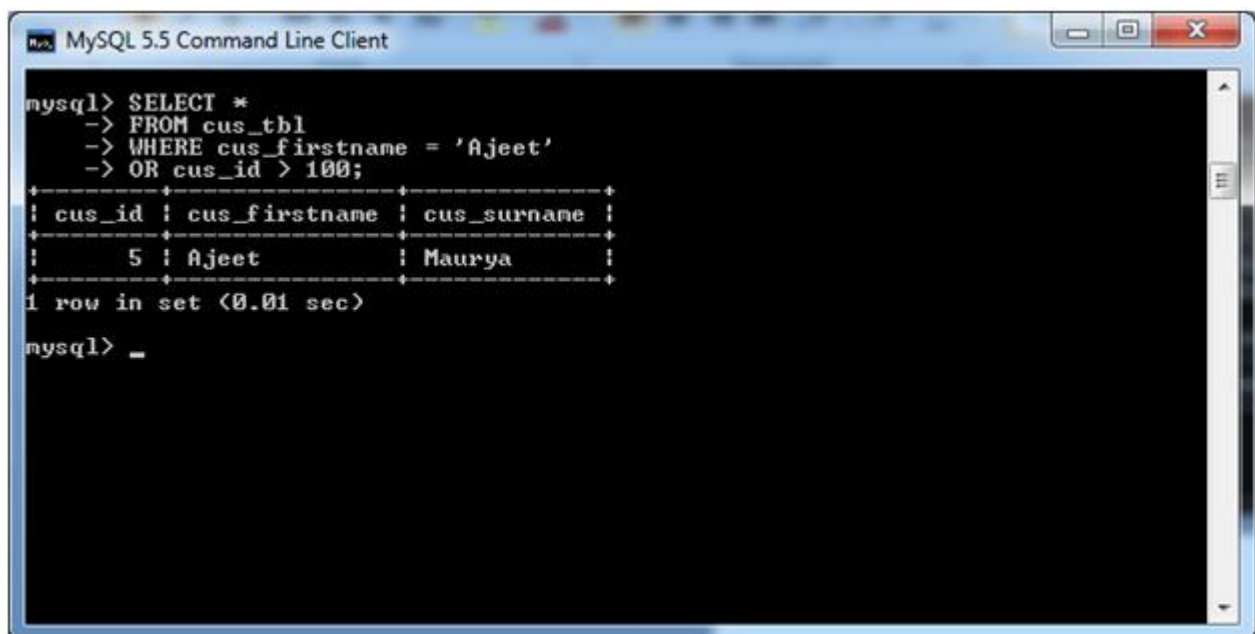
3. **WHERE** cus_firstname = 'Ajeet'
4. **AND** cus_id > 3;



```
mysql> SELECT *
-> FROM cus_tbl
-> WHERE cus_firstname = 'Ajeet'
-> AND cus_id > 3;
+-----+-----+-----+
| cus_id | cus_firstname | cus_surname |
+-----+-----+-----+
|      5 | Ajeet        | Maurya      |
+-----+-----+-----+
1 row in set (0.08 sec)

mysql>
```

1. **SELECT** * **FROM** cus_tbl
2. **WHERE** cus_firstname = 'Ajeet'
3. **OR** cus_id > 100;



```
mysql> SELECT *
-> FROM cus_tbl
-> WHERE cus_firstname = 'Ajeet'
-> OR cus_id > 100;
+-----+-----+-----+
| cus_id | cus_firstname | cus_surname |
+-----+-----+-----+
|      5 | Ajeet        | Maurya      |
+-----+-----+-----+
1 row in set (0.01 sec)

mysql> _
```

1. mysql> **CREATE TABLE** student (
2. studentid **INT PRIMARY KEY** AUTO_INCREMENT,
3. **name** **VARCHAR**(40) NOT NULL,
4. **age** **VARCHAR**(3),

5. pass BOOLEAN
6.);

Field	Type	Null	Key	Default	Extra
studentid	int	NO	PRI	NULL	auto_increment
name	varchar(40)	NO		NULL	
age	varchar(3)	YES		NULL	
pass	tinyint(1)	YES		NULL	

mysql> **INSERT INTO** student(**name**, pass) **VALUES**('Peter',true), ('John',false);

Mysql> **INSERT INTO** student(**name**, pass) **VALUES**('Miller',2);

SELECT studentid, **name**, pass **FROM** student1 **WHERE** pass **is** **TRUE**;

1. **SELECT** officer_name **FROM** officers
2. **WHERE** address LIKE 'Luck%';
3. **SELECT** officer_name **FROM** officers
4. **WHERE** address NOT LIKE 'Luck%';
5. **SELECT** * **FROM** officers
6. **WHERE** officer_name IN ('Ajeet', 'Vimal', 'Deepika');

```

mysql> SELECT *
-> FROM officers
-> WHERE officer_name IN ('Ajeet', 'Vimal', 'Deepika');
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 3 | Vimal | Faizabad |
+-----+-----+-----+
3 rows in set (0.07 sec)

mysql> _

```

1. **SELECT** * **FROM** officers
2. **WHERE** officer_name = 'Ajeet'
3. OR officer_name = 'Vimal'
4. OR officer_name = 'Deepika';

```
MySQL 5.5 Command Line Client

mysql> SELECT *
-> FROM officers
-> WHERE officer_name = 'Ajeet'
-> OR officer_name = 'Uinal'
-> OR officer_name = 'Deepika';
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 3 | Uinal | Faizabad |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

1. mysql> **SELECT** name, occupation **FROM** customer
2. **WHERE** EXISTS (**SELECT** * **FROM** Orders
3. **WHERE** customer.cust_id = Orders.cust_id);

```
MySQL 8.0 Command Line Client

mysql> SELECT name, occupation FROM customer
-> WHERE EXISTS (SELECT * FROM Orders
-> WHERE customer.cust_id = Orders.cust_id);
+-----+-----+
| name | occupation |
+-----+-----+
| Peter | Engineer |
| Joseph | Developer |
| John | Leader |
| Stephen | Scientist |
| Bob | Actor |
+-----+-----+
```

1. mysql> **DELETE FROM** Orders **WHERE** EXISTS (
2. **SELECT** * **FROM** customer
3. **WHERE** order_id=3);

```
MySQL 8.0 Command Line Client

mysql> DELETE FROM Orders WHERE EXISTS (
  -> SELECT * FROM customer
  -> WHERE order_id=3);
Query OK, 1 row affected (0.18 sec)

mysql> SELECT * FROM Orders;
+-----+-----+-----+-----+
| order_id | cust_id | prod_name | order_date |
+-----+-----+-----+-----+
|         1 |      101 | Laptop    | 2020-01-10 |
|         2 |      103 | Desktop   | 2020-02-12 |
|         4 |      104 | Mobile    | 2020-03-05 |
|         5 |      102 | TV        | 2020-03-20 |
+-----+-----+-----+-----+
```

1. **SELECT ***
2. **FROM** officers
3. **WHERE** officer_name NOT IN ('Ajeet','Vimal','Deepika');

```
MySQL 5.5 Command Line Client

3 rows in set (0.00 sec)

mysql> SELECT *
  -> FROM officers
  -> WHERE officer_name NOT IN ('Ajeet','Vimal','Deepika');
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
|         4 | Rahul        | Lucknow |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

1. **SELECT ***
2. **FROM** officers
3. **WHERE** officer_id NOT BETWEEN 3 AND 5;
4. **SELECT *** **FROM** students **WHERE** city != "England";
5. **SELECT ***
6. **FROM** officers
7. **WHERE** officer_id BETWEEN 1 AND 3;

Output:

```
MySQL 5.5 Command Line Client

mysql> SELECT *
-> FROM officers
-> WHERE officer_id BETWEEN 1 AND 3;
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 3 | Vinal | Faizabad |
+-----+-----+-----+
3 rows in set (0.06 sec)

mysql>
```

MySQL JOINS

1. **SELECT** columns
2. **FROM** table1
3. **INNER JOIN** table2
4. **ON** table1.column = table2.column;

```
MySQL 5.5 Command Line Client

4 rows in set (0.00 sec)

mysql> SELECT*FROM officers;
+-----+-----+-----+
| officer_id | officer_name | address |
+-----+-----+-----+
| 1 | Ajeet | Mau |
| 2 | Deepika | Lucknow |
| 3 | Vinal | Faizabad |
| 4 | Rahul | Lucknow |
+-----+-----+-----+
4 rows in set (0.00 sec)

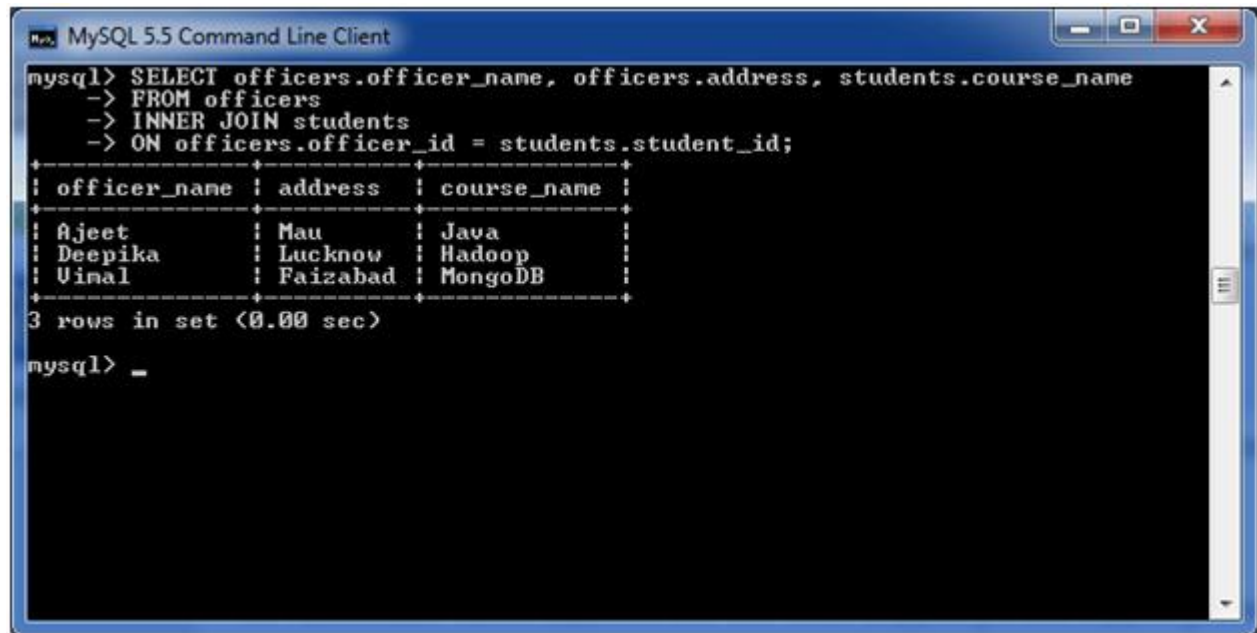
mysql> SELECT*FROM students;
+-----+-----+-----+
| student_id | student_name | course_name |
+-----+-----+-----+
| 1 | Aryan | Java |
| 2 | Rohini | Hadoop |
| 3 | Lallu | MongoDB |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

1. **SELECT** officers.officer_name, officers.address, students.course_name

2. **FROM** officers
3. **INNER JOIN** students
4. **ON** officers.officer_id = students.student_id;

Output:



```
mysql> SELECT officers.officer_name, officers.address, students.course_name
-> FROM officers
-> INNER JOIN students
-> ON officers.officer_id = students.student_id;
+-----+-----+-----+
| officer_name | address | course_name |
+-----+-----+-----+
| Ajeet       | Mau    | Java        |
| Deepika     | Lucknow | Hadoop      |
| Vinal       | Faizabad | MongoDB     |
+-----+-----+-----+
3 rows in set (0.00 sec)

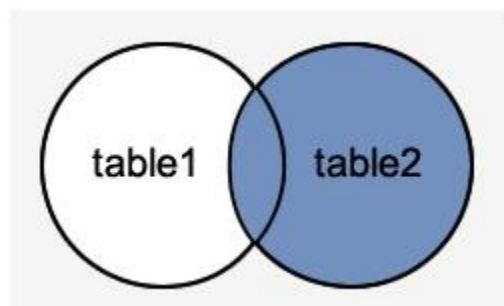
mysql> _
```

1. **SELECT** officers.officer_name, officers.address, students.course_name
2. **FROM** officers
3. **LEFT JOIN** students
4. **ON** officers.officer_id = students.student_id;

```
MySQL 5.5 Command Line Client
mysql> SELECT officers.officer_name, officers.address, students.course_name
-> FROM officers
-> LEFT JOIN students
-> ON officers.officer_id = students.student_id;
+-----+-----+-----+
| officer_name | address | course_name |
+-----+-----+-----+
| Ajeet       | Mau    | Java        |
| Deepika     | Lucknow | Hadoop      |
| Vinal       | Faizabad | MongoDB     |
| Rahul       | Lucknow | NULL        |
+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
```

MySQL Right Outer Join



1. **SELECT** officers.officer_name, officers.address, students.course_name, students.student_name
2. **FROM** officers
3. **RIGHT JOIN** students
4. **ON** officers.officer_id = students.student_id;
5. **SELECT** students.stud_fname, students.stud_lname, students.city, technologies.technology
6. **FROM** students
7. **INNER JOIN** technologies
8. **ON** students.student_id = technologies.tech_id;

```

mysql>
mysql> SELECT officers.officer_name, officers.address, students.course_name, st
students.student_name
-> FROM officers
-> RIGHT JOIN students
-> ON officers.officer_id = students.student_id;
+-----+-----+-----+-----+
| officer_name | address | course_name | student_name |
+-----+-----+-----+-----+
| Ajeet       | Mau    | Java       | Aryan       |
| Deepika     | Lucknow | Hadoop     | Rohini      |
| Uinal       | Faizabad | MongoDB   | Lallu       |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _

```

customer_id	cust_name	occupation	income	qualification
1	John Miller	Developer	20000	Btech
2	Mark Robert	Enginneer	40000	Btech
3	Reyan Watson	Scientists	60000	MSc
4	Shane Trump	Businessman	10000	MBA
5	Adam Obama	Manager	80000	MBA
6	Rincky Ponting	Cricketer	200000	Btech

order_id	date	customer_id	price
1001	2020-03-20	2	3000
1002	2020-02-15	4	2500
1003	2020-01-31	5	5000
1004	2020-03-10	2	1500
1005	2020-02-20	1	4500

1. **SELECT** customers.customer_id, cust_name, price, **date**
2. **FROM** customers
3. **LEFT JOIN** orders **ON** customers.customer_id = orders.customer_id;

customer_id	cust_name	price	date
2	Mark Robert	3000	2020-03-20
4	Shane Trump	2500	2020-02-15
5	Adam Obama	5000	2020-01-31
2	Mark Robert	1500	2020-03-10
1	John Miller	4500	2020-02-20